

LeetCode Deep Dive

From First Instinct to Optimal Algorithm: MST and Minimax Paths in Practice
Week 5 Supplement – TOAE209/TOAH209: Design and Analysis of Algorithms

Foreign Trade University

Outline

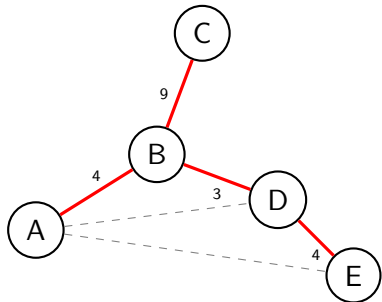
Problem statement

LeetCode 1584 – Min Cost to Connect All Points (Medium)

Given n points on a 2D plane, the cost to connect two points is their **Manhattan distance** $|x_i - x_j| + |y_i - y_j|$. Connect *all* points so that there is exactly one path between any two of them, minimizing total cost. Return the minimum total cost.

- Constraints (LeetCode): $1 \leq n \leq 1000$.
- “Exactly one path between any two points” is the definition of a **spanning tree**. Minimizing its total edge weight is exactly this week’s **minimum spanning tree** problem, on the *complete* graph where every pair of points is an edge.

Worked example



$A(0, 0)$, $B(2, 2)$, $C(3, 10)$, $D(5, 2)$, $E(7, 0)$ (not to scale).

- All $\binom{5}{2} = 10$ pairwise Manhattan distances exist as candidate edges (dashed = a sample of the ones *not* used).
- MST (red): $BD=3$, $AB=4$, $DE=4$, $BC=9$.
- **Answer:** $3 + 4 + 4 + 9 = 20$.

First instinct: enumerate spanning trees

Most direct reading of the problem

“Find the cheapest way to connect all points into a tree” $\Rightarrow$ enumerate every possible spanning tree on the n points, compute its total cost, keep the cheapest.

- By Cayley's formula, a complete graph on n labeled vertices has n^{n-2} distinct spanning trees.
- For $n = 10$: 10^8 trees. For $n = 1000$ (this problem's actual limit): 1000^{998} – a number with about 3000 digits. There is no computer, now or ever, that enumerates this.

The structural insight

- This is *exactly* the MST problem from this week's lecture, on the complete graph where every pair of points is an edge with weight equal to their Manhattan distance.
- The **cut property** says: for any partition of the points into two groups, the cheapest edge crossing that partition is safe to add. Greedy algorithms (Kruskal, Prim) exploit this *locally*, one cut at a time – no enumeration of trees is ever needed.
- The only remaining question is which greedy algorithm to run – and here the graph's **density** matters.

Choosing Kruskal vs. Prim: density matters

- The graph is **complete**: $m = \binom{n}{2} = \Theta(n^2)$ edges.
- **Kruskal** needs to generate and sort all $\Theta(n^2)$ edges first: $O(n^2 \log n)$, then Union-Find in $O(n^2 \alpha(n))$ – dominated by the sort.
- **Prim with a binary heap** is $O((m + n) \log n) = O(n^2 \log n)$ – the heap does not help here, because $m = \Theta(n^2)$ already dominates.
- **Prim with a plain array** (no heap: scan all unvisited vertices for the minimum each round) is $O(n^2)$ – strictly better, because it never needs to build or sort an edge list at all.

The implementation-choice lesson

“Use a heap” is not always the optimal move – on a *dense* graph, array-based Prim beats both Kruskal and heap-based Prim.

Optimal algorithm: array-based Prim

```
1: function MINCOSTCONNECT(points)
2:    $dist[0] \leftarrow 0$ ;  $dist[i] \leftarrow \infty$  for  $i > 0$ ;  $visited \leftarrow \emptyset$ ;  $total \leftarrow 0$ 
3:   for  $n$  rounds do
4:      $u \leftarrow \arg \min_{i \notin visited} dist[i]$ 
5:      $visited \leftarrow visited \cup \{u\}$ ;  $total \leftarrow total + dist[u]$ 
6:     for each  $v \notin visited$  do
7:        $dist[v] \leftarrow \min(dist[v], \text{MANHATTANDIST}(u, v))$ 
8:     end for
9:   end for
10:  return  $total$ 
11: end function
```

▷ scan all unvisited, $O(n)$

n rounds $\times O(n)$ work per round (scan + relax) = $O(n^2)$ total – the same “relax a distance array” idea as Dijkstra, from earlier this week.

Trace on the worked example

Start at A . $dist[\cdot]$ shown after each round's relaxation.

round	pick	cost added	running total	$dist[B]$	$dist[D]$	$dist[E]$
1	A	0	0	4	7	7
2	B	4	4	–	3	7
3	D	3	7	–	–	4
4	E	4	11	–	–	–
5	C	9	20	–	–	–

($dist[C]$ column omitted for space: it settles at $13 \rightarrow 9 \rightarrow 9 \rightarrow 9 \rightarrow 9$, picked last.) Total = 20, matching Kruskal's answer from the diagram two slides ago – confirming MST weight is independent of which correct algorithm computes it.

Complexity: three approaches

Approach	Time	Note
Enumerate all spanning trees	$O(n^{n-2})$	never finishes for $n > \sim 10$
Kruskal (sort $\Theta(n^2)$ edges)	$O(n^2 \log n)$	correct, not the fastest here
Prim, array-based	$O(n^2)$	optimal for this dense graph

Python implementation

```
1 def minCostConnectPoints(points):
2     n = len(points)
3     visited = [False] * n
4     dist = [float('inf')] * n
5     dist[0] = 0
6     total = 0
7
8     for _ in range(n):
9         u = min((d, i) for i, d in enumerate(dist) if not visited[i])[1]
10        visited[u] = True
11        total += dist[u]
12        xu, yu = points[u]
13        for v in range(n):
14            if not visited[v]:
15                xv, yv = points[v]
16                manhattan = abs(xu - xv) + abs(yu - yv)
17                if manhattan < dist[v]:
18                    dist[v] = manhattan
19    return total
```

Implementation tips & tricks (the programming side)

- **Don't build an adjacency list or edge list at all.** Since the graph is complete, Manhattan distance is computed *on the fly* from the two points' coordinates – storing $\Theta(n^2)$ edges explicitly would waste memory and time that array-based Prim never needs.
- **The unvisited-minimum scan, not a heap, is the right tool here.** `min((d, i) for i, d in enumerate(dist) if not visited[i])` is a plain $O(n)$ scan; pushing every relaxed distance into a `heapq` instead would add $O(\log n)$ per operation for no asymptotic benefit on a dense graph, while also needing “lazy deletion” of stale heap entries.
- **Unpack coordinates once per outer iteration** (`xu, yu = points[u]`), not once per inner-loop comparison – a small but real constant-factor saving repeated n times per round.

Problem statement

LeetCode 778 – Swim in Rising Water (**Hard**)

An $n \times n$ grid has elevation $grid[i][j]$ at cell (i, j) . At time t the water level is t ; you may move between two **adjacent** cells only if *both* have elevation $\leq t$. Starting at $(0, 0)$ at time 0, find the minimum t at which $(n - 1, n - 1)$ is reachable.

- Constraints (LeetCode): $1 \leq n \leq 50$, elevations are a permutation of $0, \dots, n^2 - 1$.
- Reframe: you need the path from $(0, 0)$ to $(n-1, n-1)$ that minimizes its **maximum** elevation – a **minimax path**, not a shortest (minimum-*sum*) path.

Worked example

0	2
1	3

`grid = [[0,2],[1,3]]`

- Start $(0,0)=0$, goal $(1,1)=3$.
- Every path from $(0,0)$ to $(1,1)$ must eventually step onto the cell of elevation 3 – there is no way around it.
- **Answer:** 3 – you must wait until the water (and your tolerance t) reaches 3.

First instinct: try every time step

Most direct reading of the problem

“Find the minimum t such that the goal is reachable” $\Rightarrow$ for $t = 0, 1, 2, \dots$, flood-fill (BFS/DFS) from $(0, 0)$ using only cells with elevation $\leq t$, and stop at the first t that reaches the goal.

```
1: for  $t \leftarrow 0$  to  $n^2 - 1$  do  
2:   if BFSREACHABLE( $(0, 0)$ ,  $(n-1, n-1)$ , threshold  $t$ ) then  
3:     return  $t$   
4:   end if  
5: end for
```

Why this is bad

- Each BFS/DFS flood-fill over the $n \times n$ grid costs $O(n^2)$.
- In the worst case you try $t = 0, 1, \dots, n^2 - 1$ – that is $O(n^2)$ values of t .
- Total: $O(n^2) \times O(n^2) = O(n^4)$. At $n = 50$ (the maximum), that's $50^4 = 6.25$ million – survivable here, but the pattern (“recheck everything from scratch for every candidate answer”) is the same red flag as Weeks 3–4: it will not scale, and it hides a cleaner idea.

Structural insight #1: feasibility is monotone

- If the goal is reachable at time t , it is still reachable at any later time $t' > t$ (every cell usable at t is still usable at t' , and more cells besides). Feasibility is a **monotone** yes/no function of t .
- Monotone feasibility $\Rightarrow$ **binary search on the answer**: instead of trying every t from 0 upward, binary-search t over $[0, n^2 - 1]$, checking feasibility with one BFS per guess.
- Cost: $O(\log n^2) = O(\log n)$ guesses $\times O(n^2)$ per BFS $= O(n^2 \log n)$ – already a large improvement over $O(n^4)$.

Structural insight #2: it's Kruskal's algorithm again

- An equally optimal, more elegant reformulation: process the n^2 cells in **increasing order of elevation**, “opening” each one and **union**-ing it with any already-open neighbor.
- The instant $(0, 0)$ and $(n-1, n-1)$ land in the same Union-Find component, the elevation of the cell *just opened* is the answer – the minimum bottleneck on the minimax path.
- This is exactly Week 3's Union-Find machinery and exactly this week's Kruskal's algorithm, with “edges sorted by weight” replaced by “cells sorted by elevation.” Same tool, same $O(n^2 \log n)$ bound (the sort dominates), but a single linear pass – no repeated flood-fills.

Trace on the worked example

grid = $[[0,2],[1,3]]$. Cells sorted by elevation: $(0,0)=0$, $(1,0)=1$, $(0,1)=2$, $(1,1)=3$.

open	action	start/goal connected?
$(0,0)$, elev. 0	no open neighbors yet	no
$(1,0)$, elev. 1	union with $(0,0)$	no (goal $(1,1)$ not open)
$(0,1)$, elev. 2	union with $(0,0)$	no (goal $(1,1)$ not open)
$(1,1)$, elev. 3	union with $(1,0)$ and $(0,1)$	yes $\Rightarrow$ answer = 3

Matches the expected output 3.

Complexity: naive vs. optimal

Approach	Time	At $n = 50$
Try every t + full BFS	$O(n^4)$	6.25×10^6
Binary search on t + BFS	$O(n^2 \log n)$	$\sim 2.5 \times 10^4$
Union-Find, increasing elevation	$O(n^2 \log n)$	$\sim 2.5 \times 10^4$, single pass

The last two are asymptotically tied – the Union-Find version wins on elegance and constant factors (one pass, no repeated restarts), not on Big-O.

Python implementation (part 1: setup)

```
1 def swimInWater(grid):
2     n = len(grid)
3     parent = list(range(n * n))
4
5     def find(x):
6         while parent[x] != x:
7             parent[x] = parent[parent[x]]           # path halving
8             x = parent[x]
9         return x
10
11     def union(a, b):
12         ra, rb = find(a), find(b)
13         if ra != rb:
14             parent[ra] = rb
15
16     cells_by_elevation = sorted(range(n * n), key=lambda idx: grid[idx // n][idx %
                                n])
```

Python implementation (part 2: sweep and union)

```
1  opened = [[False] * n for _ in range(n)]
2  for idx in cells_by_elevation:
3      r, c = idx // n, idx % n
4      opened[r][c] = True
5      for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
6          nr, nc = r + dr, c + dc
7          if 0 <= nr < n and 0 <= nc < n and opened[nr][nc]:
8              union(r * n + c, nr * n + nc)
9  if find(0) == find(n * n - 1):
10     return grid[r][c]
```

The elevation of the cell that *just* merged start and goal into one component is, by construction, the minimum bottleneck – no extra bookkeeping needed.

Implementation tips & tricks (the programming side)

- **Flatten (r, c) to a single index $r \cdot n + c$.** Union-Find is naturally 1-D; flattening avoids either a dictionary keyed by tuples (hashing overhead) or a 2-D parent array (extra indexing arithmetic anyway).
- **Check the stopping condition inside the sweep, not after.** Testing `find(0) == find(n*n-1)` the moment a cell is opened lets the loop return immediately – avoiding a second pass over already-processed cells.
- **Iterative find with path halving**, exactly as in Week 3 – with up to $n^2 = 2500$ cells for $n = 50$, a naive recursive find is unnecessary recursion depth risk for no benefit.
- `sorted(range(n*n), key=...)` **sorts indices, not values** – so the result directly gives you *which cell* to open next, not just its elevation.

Summary: two problems, one lesson

- 1 **Min Cost to Connect All Points** is literally this week's MST problem in disguise – the cut property makes greedy (Kruskal or Prim) correct, and *graph density* (complete graph, $m = \Theta(n^2)$) decides that plain array-based Prim, not a heap, is the fastest correct choice.
- 2 **Swim in Rising Water** reframes a minimax-path question as “add cells in increasing order until start and goal connect” – exactly Kruskal's algorithm again, with elevation playing the role of edge weight.
- 3 Both times, the naive instinct was exponential/quartic enumeration; both times, recognizing the problem as a *disguised* version of an algorithm already in this week's toolkit (MST, Union-Find) made the optimal solution almost automatic.
- 4 Implementation tips (array scan vs. heap, flattened Union-Find indices, in-loop early exit) are a second, independent layer on top of an already-optimal algorithm.

Keep practicing

LEETCODE.md lists several more Week 5 problems (*Network Delay Time*, *Connecting Cities With Minimum Cost*, *Reconstruct Itinerary*, ...) – try the same four-step process on each.